



Efficient gate reordering for Distributed Quantum Compiling in data centers

Riccardo Mengoni¹, Walter Nadalin¹, Mathys Rennela¹, Jimmy Rotureau¹, Tom Darras¹, Julien Laurat², Eleni Diamanti³ and Ioannis Lavdas^{1*}

*Correspondence:

ioannis.lavdas@weling.fr

¹Weling, 14 rue Jean Macé, 75011
Paris, France

Full list of author information is
available at the end of the article

Abstract

Just as classical computing relies on distributed systems, the quantum computing era requires new kinds of infrastructure and software tools. Quantum networks will become the backbone of hybrid, quantum-augmented data centers, in which quantum algorithms are distributed over a local network of quantum processing units (QPUs) interconnected via shared entanglement. In this context, it is crucial to develop methods and software that minimize the number of inter-QPU communications. Here we describe key features of the quantum compiler *araQne*, which is designed to minimize distribution cost, measured by the number of entangled pairs required to distribute a monolithic quantum circuit using gate teleportation protocols. In particular, we focus on the crucial role of a preprocessing stage based on gate reordering strategies, achieving a notable reduction of the distribution cost compared to a baseline approach.

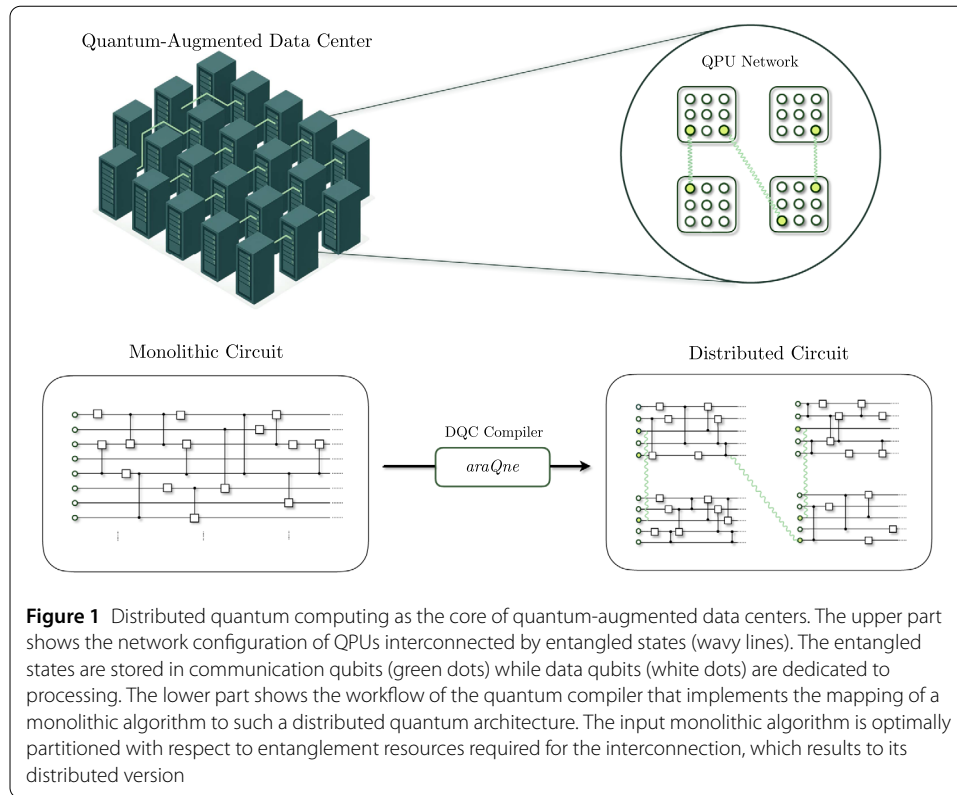
Keywords: Distributed quantum computing; Quantum circuit compilation; Modular architectures; Gate reordering; Quantum-augmented data centers

1 Introduction

Recent progress in fundamental research, hardware engineering and quantum algorithm development are bringing us closer to practical quantum computing. However, solving real-size problems and relevant industrial use cases requires quantum processing units (QPUs) with a large number of error corrected qubits [1–5]. This remains a core challenge for hardware engineers [6, 7], while quantum algorithm researchers are developing methods allowing large scale quantum computation with limited resources [8].

The concept of Distributed Quantum Computing (DQC) has emerged as a viable approach to scaling up the computational power of quantum hardware [9–11]. To realize this vision, there is a critical need for the development of dedicated software tools [12] and hardware infrastructures [13]. Quantum-augmented data centers will implement DQC by interconnecting quantum devices on a local network, in an environment tailored for quantum computation [14]. This additionally opens the door to heterogeneous DQC, which aims to harness the strengths of different QPU architectures by interconnecting processors based on different qubit modalities.

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.



At the core of the DQC approach lies a modular quantum architecture: multiple QPUs are interconnected through both classical and quantum channels, forming a network that functions as a single larger capacity quantum computer. We consider here that interconnection between QPUs is achieved through shared entanglement, as sketched in Fig. 1: a given monolithic quantum circuit is partitioned into a set of circuits executed across multiple QPUs. Within each QPU, qubits are distinguished between data qubits, dedicated to processing, and communication qubits, which store shared entangled states.

Such entangled states are expected to be scarce, as generating them on demand with high fidelity remains one of the key challenges in quantum hardware engineering [15]. For this reason, a central goal in mapping a quantum algorithm to a distributed architecture is to minimize the amount of the required inter-device entanglement. To support this objective, software tools, hereafter called compilers, can be developed to automate the translation of a quantum algorithm into an equivalent version that can run on a distributed architecture.

Several approaches to distributed compilation have been recently proposed [16–24]. Contributions include techniques where qubit assignment is determined by solving a graph partitioning problem that minimizes cross-partition edge weights. For example the approach in [19] combines graph partitioning with quantum circuit cutting, while the ones in [20] and [21] process the circuit sequentially searching for feasible qubit and gate teleportation operations. In [22] gate reordering is performed in a post-processing stage, after the qubit allocation and partitioning phases have been provided, in order to minimize quantum communication. In [16, 23], the authors first introduce the concept of distributable gate packets, which enables the circuit to be represented as a hypergraph. Then following the hypergraph partitioning, packets are subsequently refined to minimize the distribution resources. However, such method is constrained to the gate set (H ,

R_z , and CR_z) with the corresponding transpilation resulting to additional complexity. Finally, [17, 24] further extends the hypergraph-based formulation by showing how to jointly optimize the cost of gate and state teleportation to reduce overall entanglement.

Our work leverages the notion of gate packets [16, 17, 23, 24], which, combined with an efficient preprocessing circuit optimization method, yields a unique framework of distributed quantum compilation. A characterization of gate packets over a register of d qubits is provided in terms of a sequence involving $(d - 1)$ two-qubit unitaries, prior to any qubit allocation and circuit partitioning. The novel preprocessing strategy employed in this work reorders the gates of an input circuit, optimizing their grouping into gate packets. The proposed method exploits unitary operator commutation and control-symmetry of logical operations, as well as gate-packet merging, increasing the size of gate packets and thereby reducing the quantum communication overhead required for non-local operations. The above approach is shown to be effectively leveraged for distributed hypergraph-based quantum compilation.

The feature distinguishing our strategy from the above approaches, is the stage of the compilation workflow at which the circuit optimization is applied, as our method first performs allocation-agnostic gate reordering and packet construction which are subsequently given to the partitioner.

The reordering procedure is designed to be agnostic to the specific gate set used to represent the circuit, and hence avoiding complexity introduced by transpilation while enhancing its adaptability across diverse quantum hardware platforms. These design choices isolate the contribution of the reordering stage, thereby allowing us to assess how beneficial preprocessing alone can be in reducing the communication cost of a distributed computation. As a result, our framework provides a flexible and scalable solution suited for the demands of future large-scale quantum computing architectures.

The paper is organized as follows. Section 2 introduces our compiler's workflow. We develop a strategy for quantum circuit distribution, which places an emphasis on efficient gate reordering to lower the distribution cost compared to a baseline approach. We rely on a hypergraph-based framework to map monolithic circuits onto distributed circuits. This strategy has been implemented in the workflow of *araQne*, the compiler for distributed quantum computing developed by Welinq. In Sect. 3 we present results for the distribution cost of several classes of quantum circuits including circuits from the widely used QASMBench 1.4 benchmarking suite [25]. This comparison establishes the importance of efficient circuit reordering strategies for distributed quantum compiling. Detailed proofs of the methods are provided in Appendix B.

2 Compiler workflow

In the context of quantum computing, the term compilation refers to strategies that aim to simplify quantum circuits and facilitate their executions, by rewriting them into equivalent circuits with more desirable properties, such as consuming less computational resources, or being executable on specific quantum architectures.

We are interested here in the task of compiling quantum circuits for distributed quantum computers. The full workflow corresponding to those compilation routines is called a distributed quantum compiler.

In this workflow the following tasks are performed:

1. *Qubit allocation*: assigning qubits of the input monolithic circuit to the interconnected QPUs.

2. *Non-local gate scheduling*: implementing non-local gates in a way that minimizes the number of inter-QPU operations.

The above tasks are typically addressed through *circuit partitioning*, wherein a monolithic quantum circuit is decomposed into subcircuits that can be executed across a network of interconnected QPUs. An additional and crucial feature of our approach is a *circuit optimization* stage involving a set of routines aimed at minimizing the quantum resources needed for distribution.

This section presents the compilation strategies that we have implemented in our compiler's workflow. Here, the computational resource to be optimized is the *distribution cost*, that is, the number of EPR pairs consumed in distributing a monolithic quantum circuit. We have developed a greedy heuristic algorithm for gate packing: at each stage, the algorithm aims to minimize the distribution cost of the processed sub-circuit. The types of choices made by the greedy algorithm are called *choice properties*. We formalize the choice properties incorporated in our algorithm and provide a justification for their presence.

Quantum circuits are sequences of gates defined on a quantum register. Each qubit is associated to a QPU, and the resulting mapping is called an *allocation map*. For a given allocation map, the gates of a distributed circuit are either local (acting on qubits within the same QPU) or non-local (acting on qubits assigned to different QPUs). Implementing non-local gates requires inter-QPU communication. Non-local operations are implemented using the TeleGate quantum communication protocol, which consumes one pre-shared EPR pair for each execution (see Appendix A).

The overall workflow is presented schematically in Fig. 2. Following the depicted procedure, the compiler generates a distributed version of the input quantum circuit, mapping subcircuits to individual QPUs, while minimizing inter-QPU operations.

In the following, we explain how multiple quantum gates can be distributed together, as a gate packet, using a single TeleGate protocol (Sect. 2.1). We present gate reordering strategies which allow us to rewrite the input circuit reducing the distribution cost further (Sect. 2.2). All the mathematical proofs in this section can be found in Appendix B. We rely on a hypergraph-based framework to implement the distribution of the circuit, following an approach similar to [23] (Sect. 2.3).

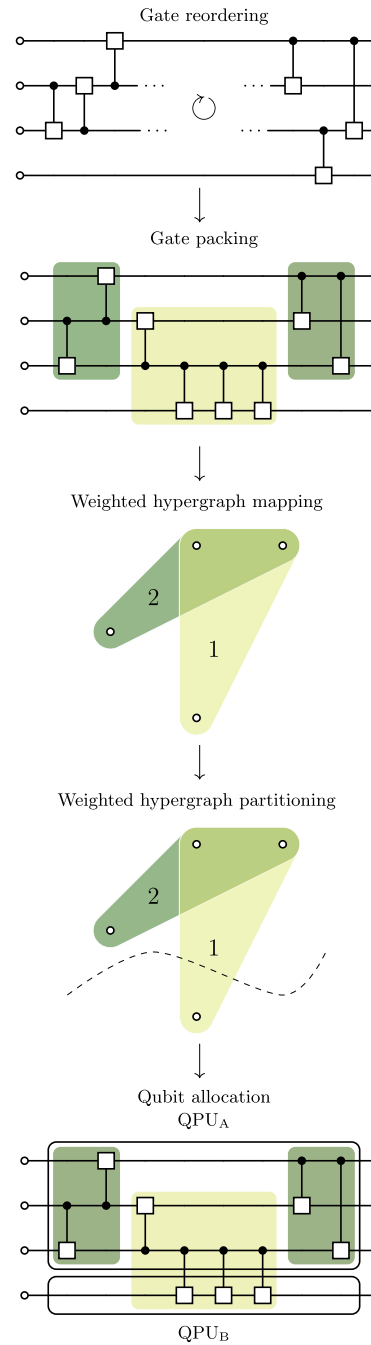
2.1 Packing quantum gates

In order to minimize the distribution cost of a given quantum circuit, one strategy is to distribute multiple quantum gates with the same quantum communication protocol. The notion of gate packet formalizes this strategy, by defining how a sequence of consecutive gates can be distributed together by the same protocol [16–18, 23, 24]. Here, we assume a universal gateset with no specific restrictions on composition, comprising arbitrary single-qubit and two-qubit gates with a matrix representation in $S = \{CU \mid U \in \mathbb{U}(2)\} \cup \mathbb{SU}(2)$, where $\mathbb{U}(2)$ and $\mathbb{SU}(2)$ are the unitary and the special unitary groups of 2×2 unitary matrices, respectively.

Definition 1 (Gate packet) Given a circuit $\mathcal{C}$, a gate packet $\mathcal{P}$ defined on a sub-register $R_{\mathcal{P}}$ and rooted on qubit $q \in R_{\mathcal{P}}$ is a sequence of consecutive gates such that:

1. each controlled-gate $g \in \mathcal{P}$ is controlled by q ,
2. each single-qubit gate $g \in \mathcal{P}$ acting on q must be either diagonal or anti-diagonal,
3. for each $t \in R_{\mathcal{P}} \setminus \{q\}$ there is at least one controlled-gate $g \in \mathcal{P}$ such that g targets t .

Figure 2 Workflow of the araQne compiler: starting from a monolithic circuit, gates are reordered to maximize gate packet size while preserving circuit equivalence (dark and light green boxes in the circuits). The circuit is then transformed into a weighted hypergraph, which is partitioned and mapped to a distributed circuit by allocating qubits to a network of QPUs. This qubit allocation strategy helps reduce the number of required EPR pairs

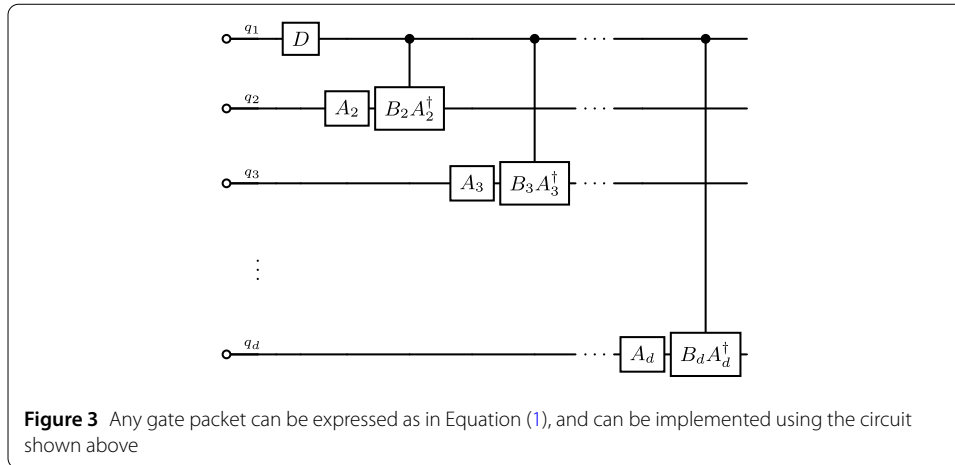


Characterization of a gate packet. The unitary operator associated to a gate packet defined on a sub-register of size d rooted on q can be written as [18]

$$(|0\rangle\langle 0| \cdot D) \otimes A + (|1\rangle\langle 1| \cdot D) \otimes B \quad (1)$$

where $D \in \{I, X\}$,

$$A = \bigotimes_{\ell=2}^d A_{\ell} \quad \text{and} \quad B = \bigotimes_{\ell=2}^d B_{\ell},$$



with $A_\ell, B_\ell \in \mathbb{U}(2)$ for each $\ell \in \{2, \dots, d\}$. Equation (1) can be visualized as a sequence of controlled gates, each rooted on the same control qubit (here labeled q_1) and having different targets as shown in Fig. 3 and proved in Appendix B.

In order to distribute a circuit $\mathcal{C}$ to a given network of quantum processors, each qubit in the register has to be assigned to a specific QPU. This process defines an allocation map.

Definition 2 (Allocation map) Let R be the register of qubits in a quantum circuit and L a set of labels associated to the available QPUs. An *allocation map* $\mathcal{A}$ is a map defined as

$$\mathcal{A}: R \rightarrow L \quad (2)$$

with $\mathcal{A}(q_i) = \ell$ if qubit q_i is assigned to QPU $_\ell$.

Consider now a gate packet $\mathcal{P}$ acting on the sub-register $R_{\mathcal{P}}$. The subset of QPUs to which the register $R_{\mathcal{P}}$ is mapped by the allocation map $\mathcal{A}$ can be expressed as $\mathcal{A}(R_{\mathcal{P}}) = \{\mathcal{A}(q) : q \in R_{\mathcal{P}}\}$. Equation (1) implies that the distribution cost of a gate packet depends solely on $|\mathcal{A}(R_{\mathcal{P}})|$, and is independent of both the number of gates within the packet and the specific qubit allocation.

Proposition 1 (Cost of distributing a gate packet) Let $\mathcal{A}$ be any allocation map. Let $\mathcal{P}$ be any gate packet defined on a sub-register $R_{\mathcal{P}}$ of size d such that $|\mathcal{A}(R_{\mathcal{P}})| = k$. Then, all gates in $\mathcal{P}$ can be implemented using not less than $k - 1$ TeleGate protocols, hence consuming $k - 1$ EPR pairs.

From there, one can easily compute the distribution cost of a circuit $\mathcal{C}$.

Definition 3 (Packing sequence) A *packing sequence* is a sequence of gate packets $\Omega_{\mathcal{C}} = (\mathcal{P}_1, \dots, \mathcal{P}_m)$ where every $g \in \mathcal{C}$ is associated to a unique $\mathcal{P} \in \Omega_{\mathcal{C}}$ and such that $\mathcal{C}$ can be obtained from $\Omega_{\mathcal{C}}$ only by adding single qubit gates between its elements.

Definition 4 (Distribution cost) Given a quantum circuit $\mathcal{C}$, a $\mathcal{C}$ packing sequence $\Omega_{\mathcal{C}}$ and an allocation map $\mathcal{A}$, the distribution cost of $\mathcal{C}$ is equal to

$$\mathcal{D}(\Omega_{\mathcal{C}}) = \sum_{\mathcal{P} \in \Omega_{\mathcal{C}}} [|\mathcal{A}(R_{\mathcal{P}})| - 1]. \quad (3)$$

Consider a quantum circuit as a sequence $\mathcal{C} = (g_1, \dots, g_m)$ of m gates acting on a n qubit register $R = \{q_1, \dots, q_n\}$, with $U_{\mathcal{C}} = g_m \cdots g_1$ being the unitary operation associated to the circuit $\mathcal{C}$.

Given that two circuits are said to be equivalent if they correspond to the same unitary transformation, it is worth noticing that equivalent circuits can lead to different packing sequences. Hence, circuit rewriting techniques can help turning the input circuit into an equivalent one that has a different packing sequence, with a lower distribution cost. In other words, one can design circuit optimization strategies for DQC. We develop such a strategy in the following section.

2.2 A greedy heuristic algorithm for gate reordering and packing

Note that two adjacent packets $\mathcal{P}_i$ and $\mathcal{P}_{i+1}$ can be merged into a single packet $\mathcal{Q}_j = \mathcal{P}_i \cdot \mathcal{P}_{i+1}$ if they are rooted on the same control qubit. We can now quantify how merging reduces the distribution cost.

Proposition 2 (Distribution cost after merging) *Given a quantum circuit $\mathcal{C}$ and its associated packet sequence $\Omega = (\mathcal{P}_1, \dots, \mathcal{P}_m)$, if $\Gamma = (\mathcal{Q}_1, \dots, \mathcal{Q}_\ell)$ with $\ell < m$ is a packing sequence constructed from Ω by merging packets then*

$$\mathcal{D}(\Gamma) \leq \mathcal{D}(\Omega)$$

for every possible qubit allocation map A .

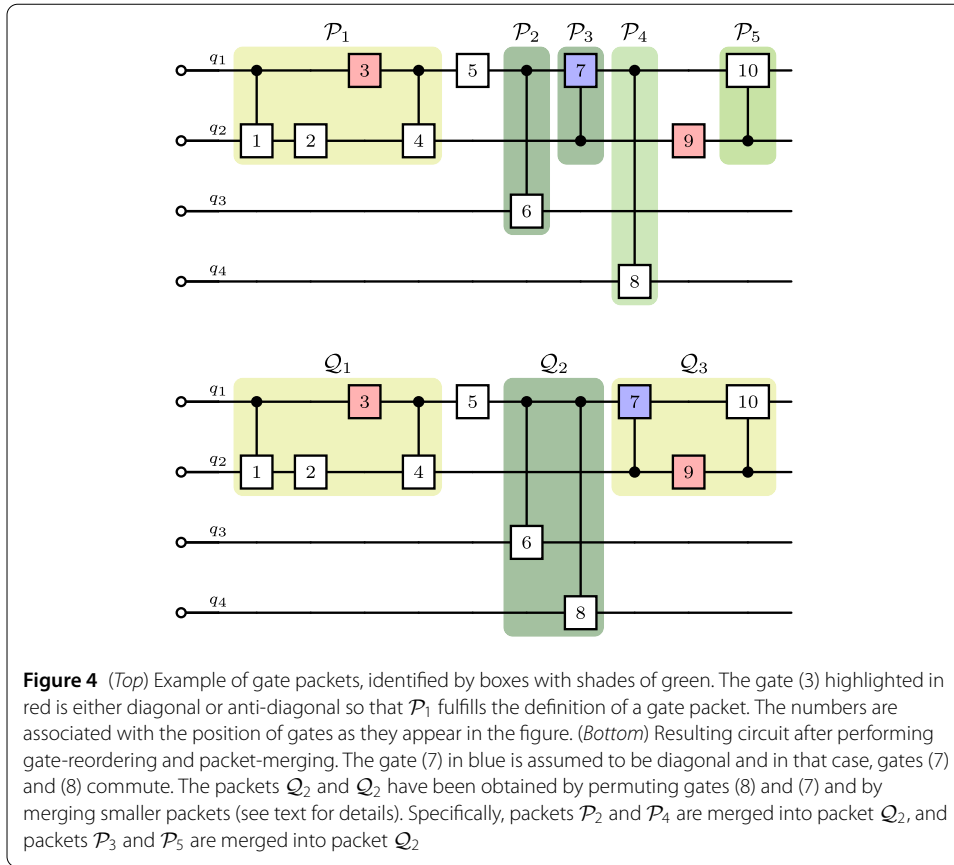
Before partitioning the input quantum circuit, it is crucial to minimize the distribution cost by merging together as many gate packets as possible and distribute them via a single TeleGate protocol. The complexity of this computational stage stems from the combinatorial growth of admissible gate reorderings generated by local commutation and packet-merging. Consequently, exhaustive global optimization is rendered practically intractable, motivating the adoption of a greedy heuristic implemented in Algorithm 1, which makes use of the following choice properties:

- *Commutativity of gates.* Given a circuit $\mathcal{C}$, if two adjacent gates g_i, g_{i+1} commute, then the unitary $U_{\mathcal{C}}$ is invariant under permutation of g_i and g_{i+1} . Algorithm 1 commutes adjacent gates whenever it helps increasing the size of gate packets, while leaving the unitary operator $U_{\mathcal{C}}$ unchanged.
- *Symmetry of controlled-gate.* A controlled- U gate CU is control-symmetric if it satisfies:

$$\begin{aligned} CU &\equiv |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U \\ &= I \otimes |0\rangle\langle 0| + U \otimes |1\rangle\langle 1|. \end{aligned}$$

A CU gate can be shown to be control-symmetric if and only if U is a phase gate $R_Z(\phi)$. Switching the control qubit of a gate packet can potentially increase the size of a gate packet.

- *Packet merging.* Consecutive gate packets can be merged together to form a larger gate packet, as long as they are rooted in the same control qubit.



We illustrate this process in Fig. 4, where the properties of our greedy approach lead to an equivalent circuit, but with a lower distribution cost. Specifically, suppose that the unitary acting on the target qubit of the control gate (7) (highlighted in blue) is diagonal. In that case, gates (7) and (8) commute and consequently they can be permuted (since the resulting circuit, after permutation, is equivalent to the initial one). After permutation of the gates, gate (8) can be added to the packet $\mathcal{P}_2$ to form a larger packet $\mathcal{Q}_2$. Assuming further that gate (9) (highlighted in red in Fig. 4) is either diagonal or anti-diagonal, the packets $\mathcal{P}_3$ and $\mathcal{P}_5$ can be merged to create the larger packet $\mathcal{Q}_3$. The worst-case running time of Algorithm 1 is $O(t m^2)$, where m is the total number of gates and t is the number of two-qubit controlled gates. Only controlled gates can start a packet, so the main outer routine is executed at most t times. For each such packet, the algorithm scans the subsequent part of the circuit, which contains at most $O(m)$ candidate gates. For each candidate gate $\bar{g}$, checking whether it can be merged with the current packet requires only constant-size information and therefore takes $O(1)$ time. The dominant cost comes from verifying whether $\bar{g}$ or the current packet $\mathcal{P}$ commutes with all gates lying between them in the circuit; in the worst case this requires scanning an interval of length $O(m)$. Therefore, the worst-case cost per packet is $O(m^2)$, and the total worst-case running time of Algorithm 1 is $O(t m^2)$.

Moreover, the commutation test is implemented through a local unitary-matrix check. Gates acting on disjoint qubit sets are treated as commuting directly. Otherwise, the two gates are represented on the joint set of qubits involved, and commutation is verified by checking whether their commutator vanishes. Previously computed commutation out-

Algorithm 1. Greedy gate reordering and packing algorithm. This algorithm creates larger gate packets using control-symmetry, commutation and packet merging properties, hence reducing the distribution cost

```

Data: circuit  $\mathcal{C}$ 
Result: packing sequence  $\Omega_{\mathcal{C}}$ 
 $\Omega_{\mathcal{C}} \leftarrow ()$ ; // Empty sequence
 $A \leftarrow \emptyset$ ;
while  $|A| \neq |\mathcal{C}|$  do
     $g \leftarrow$  first gate in  $\mathcal{C}$  which is not in  $A$ ;
    if  $g$  is a controlled-gate then
         $\mathcal{P} \leftarrow (g)$ ; // One element sequence
        foreach  $\bar{g} \in \mathcal{C}$  after  $\mathcal{P}$  do
            if  $\bar{g}$  can be merged with  $\mathcal{P}$  and  $\bar{g}$  or  $\mathcal{P}$  commutes with the gates
                between  $\mathcal{P}$  and  $\bar{g}$  then
                    rearrange  $\mathcal{C}$  to place  $\bar{g}$  and  $\mathcal{P}$  next to each other;
                     $\mathcal{P} \leftarrow \mathcal{P} + (\bar{g})$ ; // Concatenation
            end
        end
         $\Omega_{\mathcal{C}} \leftarrow \Omega_{\mathcal{C}} + (\mathcal{P})$ ;
         $A \leftarrow A \cup \{x \mid x \in \mathcal{P}\}$ ;
    end
     $A \leftarrow A \cup \{g\}$ ;
end
return  $\Omega_{\mathcal{C}}$ ;

```

comes are cached in a lookup table, so repeated occurrences are handled through an average-case $O(1)$ lookup rather than a new matrix-based computation.

In the remainder of this section, we provide further arguments justifying the relevance of our approach. For a given circuit $\mathcal{C}$, there is a gate packing sequence Ω_{min} which minimizes the distribution cost of the circuit. However, in the class of circuits equivalent to $\mathcal{C}$, there may be circuits that have gate packing sequences with distribution cost lower than $\mathcal{D}(\Omega_{min})$. One can naturally wonder then whether pairwise commutations of consecutive gates are sufficient to minimize the distribution cost among equivalent circuits.

The answer is affirmative for depth-invariant equivalent circuits. Consider the equational theory of quantum circuits developed in [26], which establishes that two circuits are equivalent if and only if they can be transformed into one another under a set of elementary circuit equivalences.

All but two of those circuit equivalences [26, equalities (n) and (o) in Fig. 3] are equivalences between circuits of different depth or can be obtained with pairwise permutations of consecutive gates.

In these two equalities, both sides are circuits with the same constraints on distributability, that is, control and target qubits have identical positions in both circuits. Therefore, applying them does not affect distributability. In conclusion, if an equivalent circuit has a gate packing sequence with a lower distribution cost than the minimal distribution cost of

the input circuit (for the same depth), it can be obtained through pairwise commutations of consecutive gates.

2.3 Weighted hypergraph mapping and partitioning

Having explained how gates can be grouped together to lower the distribution cost, we now solve the circuit partitioning problem (i.e. the problem of partitioning a circuit in sub-circuits) to be executed on different QPUs. We do so by reducing it to the hypergraph partitioning problem (see Fig. 2).

The input circuit is mapped to a hypergraph, where edges are gate packets and nodes are qubits. The hypergraph is partitioned to k -subgraphs that are then mapped to k -subcircuits, in the circuit representation. This partitioning scheme minimizes the number of cut-edges in the hypergraph, corresponding to the minimization of non-local operations between the subcircuits.

Given a packing sequence Ω_C , we map the set of qubits $R = \{q_1, \dots, q_n\}$ to the set of nodes V , and the set of packets sub-registers $\{R_{\mathcal{P}} \mid \mathcal{P} \in \Omega_C\}$ to the set of hyperedges E . An edge weighting map $w : e \mapsto w_e$ is introduced for each $e \in E$ where $w_e = |\{\mathcal{P} \in \Omega_C \mid R_{\mathcal{P}} = e\}|$ identifies the number of times that the packet subregister e appears in Ω_C .

A k -partitioning of the circuit which minimizes the distribution cost corresponds then to the k -partitioning of the hypergraph $H = (V, E, w)$. Indeed, the objective of the hypergraph k -partitioning is to find a partition $\Pi = \{V_i\}_i$, with $i \in \{1, \dots, k\}$, of V so as to minimize the cost defined as the sum:

$$\sum_{e \in E} [|F(e)| - 1] w(e)$$

where $F(e) = \{X \in \Pi \mid X \cap e \neq \emptyset\}$ is the set of parts which contain elements to which e is adjacent.

Since an allocation map $\mathcal{A}$ corresponds to a partition Π , one can see that the distribution cost is equal to the cost minimized by the hypergraph k -partitioning:

$$\begin{aligned} \mathcal{D}(\Omega_C) &= \sum_{e \in E} \sum_{\{\mathcal{P} \in \Omega_C \mid R_{\mathcal{P}} = e\}} [|A(e)| - 1] \\ &= \sum_{e \in E} [|A(e)| - 1] w_e. \end{aligned}$$

This problem is NP-hard in the balanced setting, which makes exact optimisation impractical already for moderately large instances. For this reason, we rely on KAHYPAR [27], a multilevel heuristic hypergraph partitioner known for producing very high-quality solutions and strong time-quality trade-offs in practice.

A pseudo-code that implements the explained mapping of a circuit into a weighted hypergraph is provided in Algorithm 2. Applying this algorithm to the circuit in Fig. 4, we obtain the weighted hypergraph shown in Fig. 5.

Consider now the complexity of Algorithm 2. Let m be the total number of gates in the input circuit and t the number of two-qubit controlled gates. Since Algorithm 1 creates at most one packet per controlled gate, the packet sequence Ω_C contains at most t packets. Algorithm 2 first scans Ω_C once to construct the vertex and hyperedge sets, which requires

Algorithm 2. Weighted hypergraph construction. This algorithm constructs a weighted hypergraph from a sequence of packets

Data: packing sequence Ω_C

Result: weighted hypergraph $H = (V, E, w)$

$V \leftarrow \emptyset;$

$E \leftarrow \emptyset;$

$w \leftarrow \emptyset;$

foreach $\mathcal{P}_i \in \Omega_C$ **do**

$V \leftarrow V \cup R_i;$

$E \leftarrow E \cup \{R_i\};$

end

foreach $e \in E$ **do**

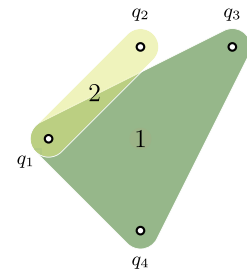
$w_e \leftarrow |\{\mathcal{P}_i \in \Omega_C \mid R_i = e\}|;$

$w \leftarrow w \cup \{w_e\};$

end

return $H = (V, E, w)$

Figure 5 Weighted hypergraph obtained from the circuit in Fig. 4. The dark green hyperedge corresponds to packet $\mathcal{Q}_2$, which is the only packet acting on qubits $\{q_1, q_3, q_4\}$, and therefore has weight 1. In contrast, the light green hyperedge has weight 2 because two packets, namely $\mathcal{P}_1$ and $\mathcal{Q}_3$, act on qubits $\{q_1, q_2\}$



$O(t)$ time. It then computes the weight of each hyperedge, since the number of distinct hyperedges is at most $|\Omega_C| \leq t$, this step requires $O(t^2)$ time in the worst case.

Assessing the ability of our algorithm to perform near-optimal choices requires to benchmark the quality of its results on standard quantum algorithms, as realized in the next section.

3 Distribution cost benchmarking

We benchmark the performance of our algorithm on several classes of quantum circuits. We suppose that the set of k QPUs forming the quantum network is made of identical QPUs with an all-to-all connectivity and where each QPU is interconnected to all other QPUs. Given a monolithic circuit of n qubits, a balanced problem is considered, where each quantum processor approximately contains $\frac{n}{k}$ qubits. To assess the performance of our approach, we first consider the case of random-generated quantum circuits. Then, we provide performance results on circuits from the QASMBench 1.4 benchmark, a widely used benchmarking suite for quantum circuits [25].

3.1 Random quantum circuits

We consider here the case of n -qubit circuits constructed as sequences of gates randomly selected from the sets of gates G_1, G_2 defined as:

$$G_1 = \{H, X, RZ(\theta)\} \quad \text{and} \quad G_2 = \{CX, CRZ(\theta)\}$$

where $\theta \in [0, 2\pi]$. This choice of gates for G_1, G_2 is motivated by the fact that (anti-)diagonal and non-diagonal single-qubit gates, as well as control-symmetric and non-control-symmetric gates are included in them. As explained in Sect. 2, (anti-)diagonal and control-symmetric gates play an important role in the construction of gate packets. As a consequence, including them in the gateset to generate random circuits provides better insights into the performance of our algorithm.

The construction of a random circuit proceeds as follows: we first randomly draw an angle θ and two qubits from a uniform probability distribution. In a next step, a gate g is randomly selected to be a one- or two-qubit gate with a probability $p = 0.2$ and $1 - p$, respectively.¹

In both cases, the selection of gates within a given set, namely G_1 or G_2 , is done uniformly at random. More explicitly, if g is a one-qubit gate, it is chosen among the gates in G_1 with equal probability for all gates. By design, if g belongs to G_1 , two gates of type g will both act on previously selected qubits. When g is a two-qubit-gate in G_2 , one of the two qubits it acts on is chosen as the control qubit uniformly at random. By construction, the process continues until n^2 gates from G_2 have been added to the circuit.

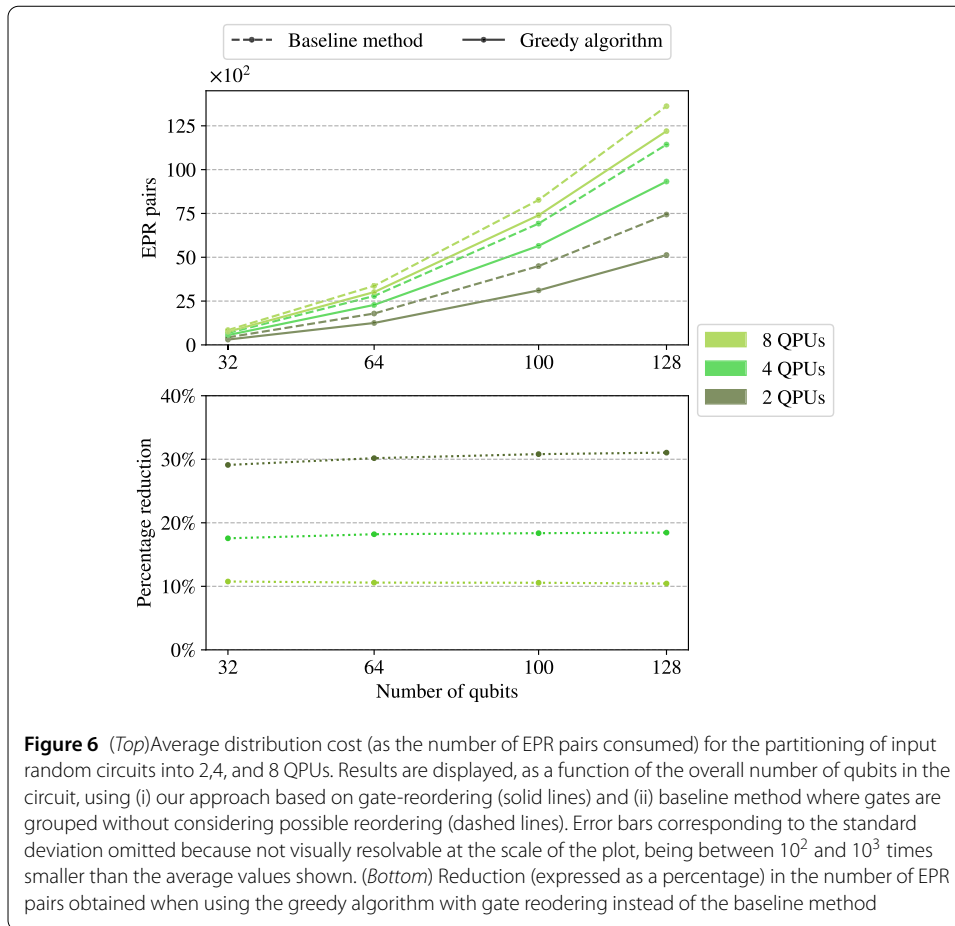
We compare the results obtained with our greedy gate-reordering approach against an ablation variant in which the same packet-construction strategy is retained, but gate reordering is disabled. In this ablated version, only consecutive controlled gates are grouped into packets. This comparison is therefore not meant as a generic baseline only, but specifically to isolate and quantify the impact of the reordering stage on the final distribution cost.

Results for the number of EPR pairs required for the partition of input circuits into 2, 4 and 8 QPUs are shown in Fig. 6 as a function of number of qubits per QPU. To avoid bias arising from the randomness in circuit construction, we average our results over 10 independent repetitions, i.e., 10 randomly generated circuits per data point.

As one can see in Fig. 6, our approach clearly allows for a reduction in the number of EPR pairs in comparison to the baseline method. We show also in the lower part of Fig. 6 the reduction, expressed as a percentage, in the number of EPR pairs obtained using the greedy approach instead of the baseline method. The results show that for a partition into 2 QPUs the average reduction is approximately 30%, for 4 QPUs, the improvement is around 19%, while for 8 QPUs, one gains a 10% reduction factor.

From the above results, the EPR pair reduction ratio obtained with our method decreases for an increasing number of QPUs. This behavior can be understood in light of the increasing complexity of the DQC problem as the number of QPUs grows. Indeed, the number of feasible gate reorderings, packet combinations, and balanced partitions increases with both the circuit size and the number of QPUs. Our greedy routine makes

¹Note that there is nothing particular about this value of p . The only criterion for fixing p is that the random circuit should include a large proportion of two-qubit gates.



locally beneficial choices, i.e., choices that enlarge the current packet or enable an immediate merge, but it is not guaranteed to solve the resulting global combinatorial optimization problem optimally. As the search space becomes larger, locally good choices are less likely to remain globally optimal, which explains why the relative gain over the no-reordering ablation decreases as the number of QPUs increases.

Finally, note that in the limit of a large number of QPUs, the distribution cost is bounded by the number of control gates in the circuit, which in this case was set to n^2 .

3.2 QASMBench quantum circuit benchmarks

We now benchmark our approach with quantum circuits from the QASMBench benchmark suite [25], in its version 1.4. Table 1 aggregates the results for the number of EPR pairs obtained for a partition into 2 QPUs using, as in the previous case of random circuits, (i) the greedy algorithm and (ii) a baseline method. For all cases considered here, computations are performed with the transpiled version of the OpenQASM circuit [25].

As seen in Table 1, the greedy algorithm allows for an overall significant reduction in the number of EPR pairs with respect to the baseline method. There are few noticeable exceptions to this pattern such as for circuits where the minimum number of EPR pairs required for inter-QPU communication, which is equal to 1 in this bipartition case, is already reached by the baseline method (see results for the Ising and KNN circuits in Table 1). Other noticeable cases include the Quantum Volume circuits [28], namely QV32,

Table 1 Distribution cost (measured as the number of EPR pairs consumed) for several circuits in the QASMBench 1.4 benchmarking suite [25]. For each circuit, its name and number of qubits n are specified. We partition all the circuits across two QPUs, each containing $\lfloor n/2 \rfloor + 1$ qubits, without making any assumptions about the intra-QPU topology. Results are shown using the greedy algorithm and a baseline implementation with no reordering. For all circuits considered here, the transpiled version was used [25]

Circuit (qubits)	Greedy algorithm	Baseline method
Adder (28)	7	9
Adder (64, 118)	7	10
BV (70, 140)	1	1
Cat (35)	1	1
DNN (33)	19	27
DNN (51)	29	42
GHZ (40)	1	1
Ising (34)	1	1
KNN (41)	1	1
Multiplier (45)	162	238
Multiplier (75)	380	568
Multiplier (350)	6397	10,954
Multiplier (400)	8319	13,492
QFT (29)	14	210
QFT (63)	36	739
QFT (160, 320)	40	820
QRAM (20)	18	33
QuGAN (39)	24	34
QuGAN (71)	38	55
QuGAN (111)	58	85
QuGAN (395)	196	292
QV (32)	600	600
QV (100)	6492	6492
Swap Test (25)	12	18
Swap Test (41)	20	30
Swap Test (83)	40	60
W-State (76, 118)	2	2
Square root (45)	3418	4419

QV100 where the greedy algorithm does not enable any reduction in the number of EPR pairs. This is naturally explained by the fact that for these circuits, commutation between gates is not possible and consequently our gate reordering procedure has no impact on results.

Table 1 includes several results for Quantum Fourier Transform (QFT) circuits with a qubit number ranging from 29 to 320. The values of rotation angles in QFT circuits can be expressed as 2^{-k} with $k \leq n$ and n the total number of qubits. Naturally, as the qubit number n increases, there are more rotation matrices with a small enough angle that can be replaced, without impacting results, by the Identity matrix.² In other words, as n increases, larger subparts of the circuit are not affected by the algorithm and consequently will not have an impact on the overall distribution cost. This results in a stagnation of the number of EPR pairs computed with the greedy algorithm at 40 (see results in Table 1 for QFT (160, 320)) for partitioning the larger QFT circuits.

In [21] the authors presented an algorithm for distributing quantum circuits based on a weighted graph description of circuits where both state and gate teleportation is considered. By comparing the results reported in Table 1 in [21], we observe that in most cases,

²our compiler uses some subroutines from Qiskit and in this context, a rotation matrix with an angle smaller than 10^{-10} is replaced by the identity matrix.

our approach allows to reach a lower or equal number of EPR pairs for a bipartition of QASM circuits. Only for the circuits Multiplier (350, 400) and the Quantum Volume QV (100), our method results in higher values for the number of EPR pairs. We want to point out, as already mentioned in previous works such as [24], that gate teleportation is not the most efficient communication protocol to distribute these circuits and that in this context, state teleportation is more suited for partitioning.

On the other hand, a comparison of our results with the ones reported in [17] and [16], (see for instance Tables 2 and 3 in [17]) show that overall, we obtain higher values of entanglement cost. This can be attributed to the fact that both works, while also relying on a hypergraph representation of the circuit, incorporate algorithmic features that are not currently implemented in our approach: in [17], both state and gate teleportation are considered prior to partitioning, whereas [16] applies embedding techniques that allow the fusion of disjoint gate packets under specific conditions, thereby reducing the distribution cost.

Under this perspective, the fact that our method performs comparably across several benchmark instances indicates that the preprocessing stage based on gate reordering can already yield meaningful reductions in distribution cost. This helps clarify the specific effect of gate reordering, independently of further refinements. At the same time, the techniques explored in the above works are best viewed as complementary to our approach, and their integration with the present preprocessing stage is a natural direction for future work.

4 Conclusion

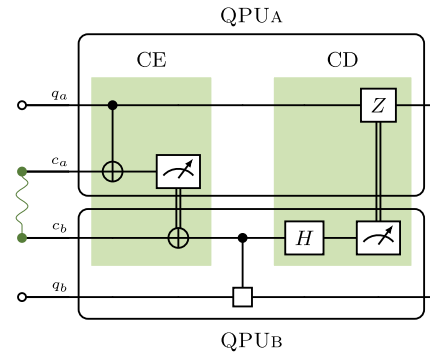
In this work, we presented a greedy heuristic algorithm for efficiently reordering gates during preprocessing, along with the packing and partitioning approaches integrated into *araQne*, the distributed quantum compiler developed by Welinq.

For an input monolithic quantum circuit, the algorithm reorders the gates and groups them into gate packets. Since all gates in a packet can be distributed using a single TeleGate protocol, this method reduces the consumption of entanglement resources. The resulting quantum circuit, expressed as a sequence of gate packets, is then mapped to a weighted hypergraph, whose hyperedges and vertices are associated with gate packets and qubits, respectively. In this way, the circuit partitioning problem is reduced to a hypergraph partitioning problem. We showed that the proposed greedy algorithm yields a significant reduction in distribution cost for random quantum circuits compared to a baseline approach where no reordering is applied. We also benchmarked our compiler on the QASMBench 1.4 benchmarking suite. Our numerical results establish the crucial role that preprocessing circuit reordering strategies play in reducing the cost of distributing quantum circuits onto interconnected QPUs.

By isolating the preprocessing stage from subsequent optimization steps, we were able to evaluate the specific contribution of gate reordering and show that it can, on its own, already yield a significant reduction in the entanglement resources involved in distributed quantum computation.

We are planning to integrate additional features to the compiler algorithm in order to enhance further its current performance and introduce hardware awareness. A wider set of communication protocols, such as TeleData and multipartite entanglement-based protocols [29], will be added, for a further reduction of the distribution cost obtained with

Figure 7 TeleGate protocol implementing a non-local gate between the data qubits q_a and q_b , which belong to distinct QPUs (A and B). The protocol incorporates the Cat-Entangler (CE) primitive, allowing for the second half of the entangled pair to act as a shared copy of the original control qubit. Finally, the shared state of the control qubit q_a is restored at QPU_A via the Cat-Disentangler (CD) primitive



TeleGate alone. In fact, depending on the compiled circuit, a choice of the use of a single or multiple protocols can be favored to minimize the distribution cost. In addition, we will consider techniques that generalize the notion of gate packets, such as embedding [16, 18]. Finally, we will introduce techniques that take into account the connectivity constraints of a given architecture, both at a global level (inter-QPU) and at a local level (coupling map), combined with the fidelity of quantum links in the distribution cost.

Appendix A: Gate teleportation

The TeleGate protocol enables the implementation of a two qubit gate involving qubits which belong to separate QPUs. As seen in Fig. 7, the Cat-Entangler (CE) allows for the second half of the entangled pair to act as a shared copy of the original control qubit. We justify the use of the TeleGate protocol in this section.

To illustrate how to perform inter-QPU quantum operations, assume that there are two qubits q_a and q_b which belong to the two distinct QPUs, respectively QPU_A and QPU_B. If a non-local controlled-gate with control qubit q_a and target qubit q_b has to be performed, then two kind of protocols can be applied: TeleData or TeleGate. The starting point of both protocols is the efficient storing of an EPR pair into two communication qubits each belonging to a distinct QPU.

In the case of the TeleData protocol, quantum teleportation transfers a qubit's state between QPUs to enable local execution of non-local operations [30]. As the data qubit from the first QPU is measured and discarded, restoring the original allocation requires a second teleportation.

On the other hand, the TeleGate protocol leverages two key communication primitives, the Cat-Entangler (CE) and Cat-Disentangler (CD) [31] to execute a non-local controlled-gate. Figure 7 illustrates one method of implementing a non-local operation using the CE and CD primitives. Unlike the TeleData protocol, only the communication qubits are measured, while the data qubits remain in their original partitions. This implies that the qubit allocation remains unchanged after applying the TeleGate protocol. Consequently, the distribution cost associated with TeleGate corresponds to the consumption of a single EPR pair. Our choice of TeleGate as a quantum communication protocol is justified further by the property that more than one gate can be distributed with a single TeleGate [32].

Appendix B: Detailed proofs

In the following, the proofs of the propositions stated in the main text are presented. These concern the distribution cost of a packet and the resulting cost after merging.

B.1 Cost of distributing a packet

Define the projector $P_i = |i\rangle\langle i|$, and the d -fold identity operator as $I_d = \bigotimes_{\ell=1}^d I$. Additionally, let $R_T \subset \{q_1, \dots, q_n\}$ be a sub-register such that $|R_T| < n$ and $q_i \notin R_T$ and $U \in \mathbb{U}(2^n)$ be a unitary operator. We define the controlled gate $CU(R_T)$, with control qubit q_i and target qubits in R_T , as the operation that applies U on R_T conditioned on the state of q_i . For simplicity, we denote this operation as CU when the control qubit and the target sub-register are clear from the context.

Proposition B.1 (Characterization of a gate packet) *The unitary operator implemented by all the gates contained in any gate packet defined on a sub-register of size d rooted on q can always be written in the following form:*

$$(|0\rangle\langle 0| \cdot D) \otimes A + (|1\rangle\langle 1| \cdot D) \otimes B$$

where $D \in \{I, X\}$, $A = \bigotimes_{\ell=2}^d A_\ell$, $B = \bigotimes_{\ell=2}^d B_\ell$, with $A_\ell, B_\ell \in \mathbb{U}(2)$ for each $\ell \in \{2, \dots, d\}$.

Proof Assume that without loss of generality, we have $R_P = \{q_1, \dots, q_d\}$ and $q = q_1$ (up to a permutation of qubit labels). Now, let us write $R_T = \{q_2, \dots, q_d\}$, and observe the following:

- any product of diagonal and anti-diagonal operators in $\mathbb{U}(2)$ is equivalent to a diagonal or anti-diagonal operator if the number of anti-diagonal operators in the product is even or odd, respectively;
- any product of single-qubit operators acting on the register $R_P \setminus \{q_0\}$ can always be written as $W = \bigotimes_{\ell=2}^d W_\ell \in \mathbb{U}(2^d)$ with $W_\ell \in \mathbb{U}(2)$;
- any product of s control gates with same control qubit is a control gate. Indeed

$$\begin{aligned} \prod_{i=1}^s CU^i(q_i) &= \prod_{i=1}^s (P_0 \otimes I_n + P_1 \otimes U^i) \\ &= P_0 \otimes I_n + P_1 \otimes \left(\prod_{i=1}^s U^i \right) \\ &= P_0 \otimes I_n + P_1 \otimes U = CU(R_T) \end{aligned}$$

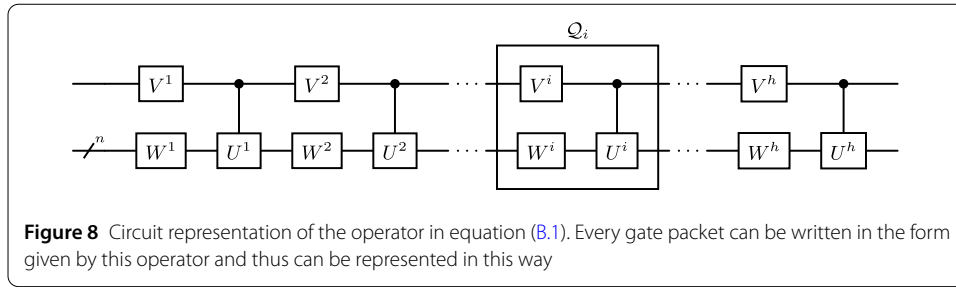
where $U^i \in \mathbb{U}(2^d)$ and

$$U = \prod_{i=1}^s \left(\bigotimes_{\ell=2}^d U_\ell^i \right) = \bigotimes_{\ell=2}^d \left(\prod_{i=1}^s U_\ell^i \right) = \bigotimes_{\ell=2}^d U_\ell$$

with $U_\ell \in \mathbb{U}(2)$, because of the gateset considered.

This means that, without loss of generality, it is always possible to write any gate packet as follows:

$$\prod_{i=1}^h CU^i \cdot (V^i \otimes W^i) = \prod_{i=1}^h \mathcal{Q}_i \quad (\text{B.1})$$



where $V^i \in \mathbb{U}(2)$ is a diagonal or anti-diagonal operator,

$$U^i = \bigotimes_{\ell=2}^d U_{\ell}^i \quad \text{and} \quad W^i = \bigotimes_{\ell=2}^d W_{\ell}^i \quad (\text{B.2})$$

with U_{ℓ}^i and W_{ℓ}^i in $\mathbb{U}(2)$, as shown in Figure 8.

First, it is proven that:

$$\mathcal{Q}_i = (P_0 \cdot D_i) \otimes L_i + (P_1 \cdot D_i) \otimes B_i \quad (\text{B.3})$$

with $D_i \in \{I, X\}$. In order to do that, two cases have to be considered.

1. V^i is diagonal, that is

$$V^i = \begin{bmatrix} L_i & 0 \\ 0 & b_i \end{bmatrix} = L_i P_0 + b_i P_1.$$

In this case it is possible to write

$$\begin{aligned} \mathcal{Q}_i &= (P_0 \cdot I) \otimes (L_i W^i) + (P_1 \cdot I) \otimes (b_i U^i \cdot W^i) \\ &= (P_0 \cdot D_i) \otimes L_i + (P_1 \cdot D_i) \otimes B_i \end{aligned}$$

where $L_i = L_i W^i$, $B_i = b_i U^i \cdot W^i$ and $D_i = I$.

2. V^i is anti-diagonal, i.e.:

$$V^i = \begin{bmatrix} 0 & L_i \\ b_i & 0 \end{bmatrix} = L_i |0\rangle\langle 1| + b_i |1\rangle\langle 0|.$$

In this case, noticing that $P_0 \cdot X = |0\rangle\langle 1|$ and $P_1 \cdot X = |1\rangle\langle 0|$, it is possible to write

$$\begin{aligned} \mathcal{Q}_i &= (P_0 \cdot X) \otimes (L_i W^i) + (P_1 \cdot X) \otimes (b_i U^i \cdot W^i) \\ &= (P_0 \cdot D_i) \otimes L_i + (P_1 \cdot D_i) \otimes B_i \end{aligned}$$

where $L_i = L_i W^i$, $B_i = b_i U^i \cdot W^i$ and $D_i = X$.

Now we show that

$$Q_i \cdot Q_j = (P_0 \cdot D) \otimes \alpha + (P_1 \cdot D) \otimes \beta \quad (\text{B.4})$$

by distinguishing the following two cases.

1. $D_i = I$. In this case, it is possible to write the composition of two consecutive operators Q_i and Q_j as

$$\begin{aligned} Q_i \cdot Q_j &= (P_0 \cdot D_j) \otimes (L_i \cdot A_j) + (P_1 \cdot D_j) \otimes (B_i \cdot B_j) \\ &= (P_0 \cdot D) \otimes \alpha + (P_1 \cdot D) \otimes \beta \end{aligned}$$

where $C = D_j \in \{I, X\}$, $\alpha = L_i \cdot A_j$ and $\beta = B_i \cdot B_j$.

2. $D_i = X$. In this case

$$\begin{aligned} Q_i \cdot Q_j &= (|0\rangle\langle 1| \cdot D_j) \otimes (L_i \cdot B_j) + (|1\rangle\langle 0| \cdot D_j) \otimes (B_i \cdot A_j) \\ &= (P_0 \cdot D) \otimes \alpha + (P_1 \cdot D) \otimes \beta \end{aligned}$$

where $D = I$ if $D_j = X$ and $D = X$ if $D_j = I$, $\alpha = L_i \cdot B_j$ and $\beta = B_i \cdot A_j$.

Equations (B.3) and (B.4) imply that any assignment packet can always be written as:

$$(P_0 \cdot D) \otimes A + (P_1 \cdot D) \otimes B$$

with $C \in \{I, X\}$ and $A, B \in \mathbb{U}(2^d)$.

Finally, the fact that

$$A = \bigotimes_{\ell=2}^d A_\ell \quad \text{and} \quad B = \bigotimes_{\ell=2}^d B_\ell,$$

with $A_\ell, B_\ell \in \mathbb{U}(2)$, is an immediate consequence of equation (B.2). $\square$

From this proposition, the following corollary follows:

Proposition B.2 (Cost of distributing a gate packet) *Let $\mathcal{A}$ be any allocation map. Let $\mathcal{P}$ be any gate packet defined on a sub-register $R_{\mathcal{P}}$ of size d such that $|\mathcal{A}(R_{\mathcal{P}})| = k$. Then, all gates in $\mathcal{P}$ can be implemented using not less than $k - 1$ TeleGate protocols, hence consuming $k - 1$ EPR pairs.*

Proof We proceed by induction on $k = |\mathcal{A}(R_{\mathcal{P}})|$.

For $k = 1$, no EPR pair is required to implement local gates that act on qubits managed by the same QPU. For $k = 2$, the gate packet $\mathcal{P}$ satisfies the distributability condition introduced in [18] and thus it is possible to distribute it using a single EPR pair.

When $k > 2$, it is possible to assume that the sub-register $R_a = \{q_{n_{a-1}}, q_{n_{a-1}+1}, \dots, q_{n_a-1}, q_{n_a}\}$ with $n_{a-1} < n_a$ is allocated to QPU a for each $a \in \{1, \dots, k\}$, up to a permutation of the qubit and QPU indices. Introducing $n_0 = 0$, noticing that $n_k = d$, and using the Prop. B.1, the unitary implementing $\mathcal{P}$ can be written as

$$(P_0 \cdot D) \otimes \left(\bigotimes_{\ell=2}^d A_\ell \right) + (P_1 \cdot D) \otimes \left(\bigotimes_{\ell=2}^d B_\ell \right) = \left(\prod_{a=1}^k \mathcal{P}_a \right) \cdot (D \otimes I_n) \quad (\text{B.5})$$

where

$$\begin{aligned}\mathcal{P}_a &= P_0 \otimes I_{n_{a-1}} \otimes \left(\bigotimes_{\ell=n_{a-1}}^{n_a} A_\ell \right) \otimes I_{d-n_a} + P_1 \otimes I_{n_{a-1}} \otimes \left(\bigotimes_{\ell=n_{a-1}}^{n_a} B_\ell \right) \otimes I_{d-n_a} \\ &= P_0 \otimes I_{n_{a-1}} \otimes \alpha_a \otimes I_{d-n_a} + P_1 \otimes I_{n_{a-1}} \otimes \beta_a \otimes I_{d-n_a}.\end{aligned}$$

It is then possible to consider $\mathcal{P}$ as a sequence of k packets rooted on q_1

$$\mathcal{P} = (C \otimes I_n, \mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_k)$$

where $|\mathcal{A}(R_{\mathcal{P}_1})| = 1$ and $|\mathcal{A}(R_{\mathcal{P}_a})| = 2$ for each $a \in \{2, \dots, k\}$. Therefore, every $\mathcal{P}_a$ with $a > 2$ can be distributed using a single EPR pair, which implies that $\mathcal{P}$ can be executed using $k - 1$ EPR pairs. $\square$

B.2 Distribution cost after merging

Proposition B.3 (Distribution cost after merging) *Given a quantum circuit $\mathcal{C}$ and its associated packet sequence $\Omega = (\mathcal{P}_1, \dots, \mathcal{P}_m)$, if $\Gamma = (\mathcal{Q}_1, \dots, \mathcal{Q}_\ell)$ with $\ell < m$ is a packing sequence constructed from Ω by means of merging packets then*

$$\mathcal{D}(\Gamma) \leq \mathcal{D}(\Omega)$$

for each qubit allocation map $\mathcal{A}$.

Proof Let $\mathcal{P}_i$ and $\mathcal{P}_{i+1}$ two adjacent packets that can be merged into a packet $\mathcal{Q}_j = \mathcal{P}_i \cdot \mathcal{P}_{i+1}$, with q being the root control qubit. We distinguish the following two cases.

- $\mathcal{A}(R_{\mathcal{P}_i}) \cap \mathcal{A}(R_{\mathcal{P}_{i+1}}) \supsetneq \mathcal{A}(q)$. In this case it follows that

$$\begin{aligned}|\mathcal{A}(R_{\mathcal{Q}_j})| &< |\mathcal{A}(R_{\mathcal{Q}_j})| + |\mathcal{A}(R_{\mathcal{P}_i}) \cap \mathcal{A}(R_{\mathcal{P}_{i+1}}) \setminus \mathcal{A}(q)| \\ &= |\mathcal{A}(R_{\mathcal{P}_i})| + |\mathcal{A}(R_{\mathcal{P}_{i+1}})|.\end{aligned}$$

Indeed, only one TeleGate is needed to perform a non-local operation involving a common allocated QPU different from $\text{QPU}_{\mathcal{A}(q)}$, instead of two. Hence in this case $\mathcal{D}(\Gamma) < \mathcal{D}(\Omega)$.

- $\mathcal{A}(R_{\mathcal{P}_i}) \cap \mathcal{A}(R_{\mathcal{P}_{i+1}}) = \mathcal{A}(q)$. In this case it follows that

$$|\mathcal{A}(R_{\mathcal{Q}_j})| = |\mathcal{A}(R_{\mathcal{P}_i})| + |\mathcal{A}(R_{\mathcal{P}_{i+1}})|.$$

Hence there is no advantage in merging because $\mathcal{D}(\Gamma) = \mathcal{D}(\Omega)$.

Since Γ is a packing sequence constructed from Ω only by merging packets then the proposition follows. $\square$

Acknowledgements

This research has received funding from the Région Ile-de-France as part of the project AQADOC (Convention No. 24003067), from the European Innovation Council (EIC) Accelerator under Grant Agreement No. 101188682 for the project SQOUT and from the project Hybrid HPC Quantum Initiative (HQI) which is part of Plan France 2030. J.L. is a member of the Institut Universitaire de France.

Author contributions

RM, WN, MR, JR, and IL have contributed equally to the development of the article, including content creation, calculations, coding and figure generation. TD, JL, and ED designed the research and supervised the project. All authors discussed the results and contributed to the writing of the manuscript.

Data availability

No datasets were generated or analysed during the current study.

Declarations**Competing interests**

TD, JL, ED are cofounders and shareholders of Welingq.

Author details

¹Welingq, 14 rue Jean Macé, 75011 Paris, France. ²Laboratoire Kastler Brossel, Sorbonne Université, CNRS, ENS-Université PSL, Collège de France, 4 Place Jussieu, 75005 Paris, France. ³LIP6, Sorbonne Université, CNRS, 4 Place Jussieu, 75005 Paris, France.

Received: 4 December 2025 Accepted: 15 April 2026 Published online: 21 April 2026

References

1. Orús R, Muga S, Lizaso E. Quantum computing for finance: overview and prospects. *Rev Phys.* 2019;4:100028.
2. Dalzell AM, McArdle S, Berta M, et al. Quantum algorithms: a survey of applications and end-to-end complexities. [arXiv:2310.03011](https://arxiv.org/abs/2310.03011).
3. Núñez-Merino M, Maqueira-Marín JM, et al. Quantum-inspired computing technology in operations and logistics management. *Int J Phys Distrib Logist Manag.* 2024;54:247.
4. Leclerc L, Dalyac C, Bendotti P, et al. Implementing transferable annealing protocols for combinatorial optimization on neutral-atom quantum processors: a case study on smart charging of electric vehicles. *Phys Rev A.* 2025;111.
5. Cazals P, François A, Henriot L, et al. Identifying hard native instances for the maximum independent set problem on neutral atoms quantum processors. [arXiv:2502.04291](https://arxiv.org/abs/2502.04291) [quant-ph].
6. Mohseni M, Scherer A, Johnson KG, et al. How to build a quantum supercomputer: scaling from hundreds to millions of qubits. [arXiv:2411.10406](https://arxiv.org/abs/2411.10406).
7. Sinclair J, Ramette J, Grinkemeyer B, et al. Fault-tolerant optical interconnects for neutral-atom arrays. *Phys Rev Res.* 2025;7:013313.
8. Barral D, Cardama FJ, Díaz-Camacho G, et al. Review of distributed quantum computing: from single qpu to high performance quantum computing. *Comput Sci Rev.* 2025;57:100747.
9. Caleffi M, Amoretti M, Ferrari D, et al. Distributed quantum computing: a survey. *Comput Netw.* 2024;254:110672.
10. Liu J, Hann CT, Jiang L. Data centers with quantum random access memory and quantum networks. *Phys Rev A.* 2023;108:032610.
11. Main D, Drmota P, Nadlinger DP, Ainley EM, Agrawal A, Nichol BC, Srinivas R, Araneda G, Lucas DM. Distributed quantum computing across an optical network link. *Nature.* 2025;638:383.
12. Ferrari D, Cacciapuoti AS, Amoretti M, et al. Compiler design for distributed quantum computing. *IEEE Trans Quantum Eng.* 2021;2:1.
13. Weaver MJ, Duivesteyn P, Bernasconi AC, et al. An integrated microwave-to-optics interface for scalable quantum computing. *Nat Nanotechnol.* 2024;19:166.
14. Shapourian H, Kaur E, Sewell T, et al. Quantum data center infrastructures: a scalable architectural design perspective. [arXiv:2501.05598](https://arxiv.org/abs/2501.05598).
15. Ang J, Carini G, Chen Y, et al. Arquin: architectures for multinode superconducting quantum computers. *ACM Trans Quantum Comput.* 2024;5.
16. Andrés-Martínez P, Forrer T, Mills D, et al. Distributing circuits over heterogeneous, modular quantum computing network architectures. *Quantum Sci Technol.* 2024;9:045021.
17. Burt F, Chen K-C, Leung KK. A multilevel framework for partitioning quantum circuits. 2025. [arXiv:2503.19082](https://arxiv.org/abs/2503.19082) [quant-ph].
18. Wu J-Y, Matsui K, Forrer T, et al. Entanglement-efficient bipartite-distributed quantum computing. *Quantum.* 2023;7:1196.
19. Tomesh T, Saleem ZH, Perlin MA, et al. Divide and conquer for combinatorial optimization and distributed quantum computation. In: 2023 IEEE international conference on quantum computing and engineering (QCE). vol. 01. 2023. p. 1–12.
20. Ferrari D, Carretta S, Amoretti M. A modular quantum compilation framework for distributed quantum computing. *IEEE Trans Quantum Eng.* 2023;4:1.
21. Kaur E, Shapourian H, Zhao J, et al. Optimized quantum circuit partitioning across multiple quantum processors. In: Hemmer PR, Migdall AL, editors. Quantum computing, communication, and simulation V. vol. 13391. International Society for Optics and Photonics (SPIE). 2025. p. 133910L.
22. Wu A, Zhang H, Li G, et al. Autocomm: a framework for enabling efficient communication in distributed quantum programs. In: 2022 55th IEEE/ACM international symposium on microarchitecture (MICRO). 2022. p. 1027–41.
23. Andrés-Martínez P, Heunen C. Automated distribution of quantum circuits via hypergraph partitioning. *Phys Rev A.* 2019;100:032308.
24. Burt F, Chen K-C, Leung KK. Generalised circuit partitioning for distributed quantum computing. In: 2024 IEEE international conference on quantum computing and engineering (QCE). vol. 02. 2024. p. 173–8.
25. Li A, Stein S, Krishnamoorthy S, et al. Qasmbench: a low-level quantum benchmark suite for nisy evaluation and simulation. *ACM Trans Quantum Comput.* 2023;4.

26. Clément A, Heurtel N, Mansfield S, et al. A complete equational theory for quantum circuits. In: 2023 38th annual ACM/IEEE symposium on logic in computer science (LICS). 2023. p. 1–13.
27. Schlag S, Heuer T, Gottesbüren L, Akhremtsev Y, Schulz C, Sanders P. 2021. High-quality hypergraph partitioning. [arXiv:2106.08696](https://arxiv.org/abs/2106.08696) [cs.DS].
28. Cross AW, Bishop LS, Sheldon S, et al. Validating quantum computers using randomized model circuits. *Phys Rev A*. 2019;100:032328.
29. Riera-Sàbat F, Dür W. A modular entanglement-based quantum computer architecture. *New J Phys*. 2024;26:123015.
30. Sundaram RG, Gupta H. Distributing quantum circuits using teleportations. In: 2023 IEEE international conference on quantum software (QSW). 2023. p. 186–92.
31. Eisert J, Jacobs K, Papadopoulos P, et al. Optimal local implementation of nonlocal quantum gates. *Phys Rev A*. 2000;62.
32. Ghaderibaneh M, Gupta H, Ramakrishnan C. Generation and distribution of ghz states in quantum networks. In: 2023 IEEE international conference on quantum computing and engineering (QCE). vol. 01. 2023. p. 1120–31.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.